

Reviewer Pack — Run-Length Entropy Floor Lower-Bounds DNF-MCSP Term Count

Entry 702dd618723f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 18:40:55 UTC

Run-Length Entropy Floor Lower-Bounds DNF-MCSP Term Count

Entry ID: 702dd618723f **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 18:40:55 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics on words: run-length encoding (RLE) entropy of the truth-table string, specifically $H_{\text{RLE}}(f) = -\sum (l_i/2^n) \log_2(l_i/2^n)$ where l_i are the maximal constant-block lengths in the lex-ordered truth table $\text{TT}(f) \in \{0,1\}^{2^n}$ (a Lempel-Ziv-adjacent but distinct word statistic, rarely used in meta-complexity) **Field B** (complexity object): DNF-MCSP: minimum number of product terms $\text{DNF}_{\min}(f)$ for a Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$ given its 2^n -bit truth table; the Hirahara-Oliveira-Santhanam restricted-MCSP proxy for circuit-MCSP

Statement:

For every Boolean function f on $n \leq 6$ variables with at least one satisfying and one falsifying assignment, $\text{DNF}_{\min}(f) \geq \lceil 2^{\{H_{\text{RLE}}(f)\}} / (n+1) \rceil$, where $H_{\text{RLE}}(f)$ is the run-length Shannon entropy (in bits) of the lex-ordered truth-table word. Equivalently, $\exp_2(H_{\text{RLE}}(f)) \leq (n+1) \cdot \text{DNF}_{\min}(f)$ for all such f . A single counterexample — any f with $(n+1) \cdot \text{DNF}_{\min}(f) < 2^{\{H_{\text{RLE}}(f)\}}$ — refutes the conjecture.

Rationale (proposer's reasoning):

$\text{DNF}_{\min}(f)$ is bounded by the number of maximal monochromatic subcubes covering $\text{supp}(f)$, and each such subcube contributes a contiguous run in $\text{TT}(f)$ of length 2^k for some $k \leq n$ only along the lowest-indexed coordinate, so RLE block-structure is forced by DNF cover structure; converting the multiplicative lex-ordering bias into Shannon entropy yields a sub-linear-in- 2^n floor that is non-trivial for structured f and conjecturally tight up to the $(n+1)$ Boolean-fan-in slack. The bridge — RLE entropy of the truth-table word as an MCSP lower bound — sits in the Hirahara-Oliveira-Santhanam meta-complexity program and does not relativize because it directly inspects the truth-table representation. RLE entropy is computable in $O(2^n)$ time, side-stepping the ‘uncomputable invariant’ failure mode that killed earlier symbolic-dynamics attempts.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2671706b8c46f101

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 800 Boolean functions ($n \in \{3, 4, 5, 6\}$; 200 random at each density $\{1/4, 1/2, 3/4\}$ plus 100 structured DNFs per n), compute slack $S(f) = (n+1) \cdot \text{DNF_min}(f) - 2^{\{H_{\text{RLE}}(f)\}}$. SUPPORTED iff $S(f) \geq 0$ on $\geq 99\%$ of samples AND median $S(f)$ is non-decreasing across $n=3 \rightarrow 6$. FALSIFIED on any single seed with $S(f) < 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - DNF minimization truth table run-length encoding entropy lower bound - minimum DNF term count Boolean function Lempel-Ziv complexity truth table - MCSP minimum circuit size DNF complexity word entropy combinatorics

Top relevant hits considered: - [<http://arxiv.org/abs/2511.07739v3>] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1802.00703v2>] From Clustering Supersequences to Entropy Minimizing Subsequences for Single and Double Deletions - [<http://arxiv.org/abs/1501.01331v1>] DNF complexity of complete boolean functions - [<http://arxiv.org/abs/1311.0822v1>] Properties of maximum Lempel-Ziv complexity strings - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1401.6294v1>] An Extended Result on the Optimal Estimation under Minimum Error Entropy Criterion

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import Counter

def run_trial(seed: int) -> dict:
    n = random.choice([3, 4, 5, 6])
    density = random.choice([1/4, 1/2, 3/4])
    num_samples = 200
    num_structured = 100

    def generate_random_function(n, density):
        tt = [random.randint(0, 1) for _ in range(2**n)]
```

```

    for i in range(len(tt)):
        if random.random() < density:
            tt[i] = 1 - tt[i]
    return tt

def generate_structured_function(n, t):
    terms = []
    for _ in range(t):
        term = [random.randint(0, 1) for _ in range(n)]
        terms.append(term)
    return terms

def rle_encoding(tt):
    encoded = []
    count = 1
    prev = tt[0]
    for i in range(1, len(tt)):
        if tt[i] == prev:
            count += 1
        else:
            encoded.append((prev, count))
            prev = tt[i]
            count = 1
    encoded.append((prev, count))
    return encoded

def h_rle(tt):
    rle = rle_encoding(tt)
    entropy = sum(count * math.log2(length + 1) for _, length in rle)
    return entropy

def dnf_min(tt):
    n = int(math.log2(len(tt)))
    prime_implicants = []

    def is_prime_implicant(pi, tt):
        for i in range(len(tt)):
            if pi[i] == 0 and tt[i] == 1:
                return False
        return True

    def cover(tt, pi):
        covered = [False] * len(tt)
        for i in range(len(tt)):
            if all(pi[j] == tt[i][j] or pi[j] == 0 for j in range(n)):
                covered[i] = True
        return all(covered)

    for i in range(2**n):
        pi = [i >> j & 1 for j in range(n)]
        if is_prime_implicant(pi, tt):
            prime_implicants.append(pi)

    min_cover_size = float('inf')
    for i in range(1 <= len(prime_implicants)):
        cover_set = [prime_implicants[j] for j in range(len(prime_implicants)) if (i >> j) & 1]

```

```

        if cover(tt, cover_set):
            min_cover_size = min(min_cover_size, sum(pi.count(1) for pi in cover_set))

    return min_cover_size

def compute_slack(tt):
    h_rle_val = h_rle(tt)
    dnf_min_val = dnf_min(tt)
    slack = (n + 1) * dnf_min_val - 2**h_rle_val
    return slack

random_functions = [generate_random_function(n, density) for _ in range(num_samples)]
structured_functions = [generate_structured_function(n, t) for t in range(1, 9)]

all_slacks = []
for tt in random_functions + structured_functions:
    slack = compute_slack(tt)
    all_slacks.append(slack)

mean_slack = sum(all_slacks) / len(all_slacks)
median_slack = sorted(all_slacks)[len(all_slacks) // 2]

conjecture_holds = all(slack >= 0 for slack in all_slacks)
counterexample = "" if conjecture_holds else f"Slack < 0: {min(all_slacks)}"

return {
    "metric_name": "slack",
    "metric_value": mean_slack,
    "instances_tested": len(all_slacks),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        random.seed(seed)
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_slack = sum(r["metric_value"] for r in results) / len(results)
    median_slack = sorted([r["metric_value"] for r in results])[len(results) // 2]
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slack} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Slack < 0\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0742e7bb.py", line 121, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0742e7bb.py", line 97, in run_trial
    slack = compute_slack(tt)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0742e7bb.py", line 87, in compute_slack
    h_rle_val = h_rle(tt)
               ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0742e7bb.py", line 53, in h_rle
    entropy = sum(count * math.log2(length + 1) for _, length in rle)
               ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0742e7bb.py", line 53, in <genexpr>
    entropy = sum(count * math.log2(length + 1) for _, length in rle)
               ~~~~~
```

NameError: name 'count' is not defined. Did you mean: 'Counter'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError in the h_rle function (undefined variable 'count' in the generator expression) before any trials produced data, so the pre-registered support condition could not be evaluated. | next: Fix the h_rle implementation (e.g., use Counter over run lengths or iterate as 'for length, count in counter.items()') and rerun the ≥800-sample sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	21822
2	preregistration	claude_max	opus	0	0	7631
3	novelty	claude_max	opus	0	0	9933
4	novelty	claude_max	opus	0	0	8707
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16957
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13216

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	judge	claude_max	opus	0	0	4544

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 82810 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/702dd618723f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/702dd618723f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/702dd618723f.tar.gz (if generated)